

Animal Shelter Simulator - Project 3 Design Document

Owen Lindsey
Professor David Parker
CST-180: Python Programming 1
Grand Canyon University
December 11, 2025

***[Video Link](#)

***[src code](#)

Summary

This project adds a **Shelter** management system that handles the complete animal care workflow. The implementation uses object-oriented programming principles including class inheritance, composition, capacity constraints, and workflow automation. The shelter manages kennel allocation, tracks adoption requests through a waitlist system, and prevents overcrowding by reusing empty kennels.

Key Features

- **Kennel Management:** Reuses empty kennels before creating new ones
 - **Capacity Limits:** Prevents overcrowding with configurable capacity
 - **Adoption System:** Matches adopters with available animals and frees kennels
 - **Waitlist:** Records adoption requests when animals aren't available
 - **Type Safety:** Uses Python's typing system for better code clarity
-

System Architecture

Input / Processing / Output Overview

Stage	Description
Input	Animal details (name , age , type-specific attribute); shelter capacity; adopter requests specifying an animal type and optional adopter name for the waitlist.

Stage	Description
Processing	Constructors validate data, shelters locate empty kennels, and adoption paths either free kennels or log waitlist entries. Business rules enforce one animal per kennel, automatic kennel creation up to the set capacity, and waitlist tracking for unavailable types.
Output	String renderings of animals, kennels, and shelter status; Boolean success/failure for intake; adopted animal objects (or <code>None</code> when unavailable); waitlist snapshots for each animal type.

Class Hierarchy and Relationships

Inheritance Structure

The code uses an object-oriented inheritance hierarchy with three animal types that inherit from a base class:

```
Animal (Base Class)
├── Dog
├── Cat
└── Bird
```

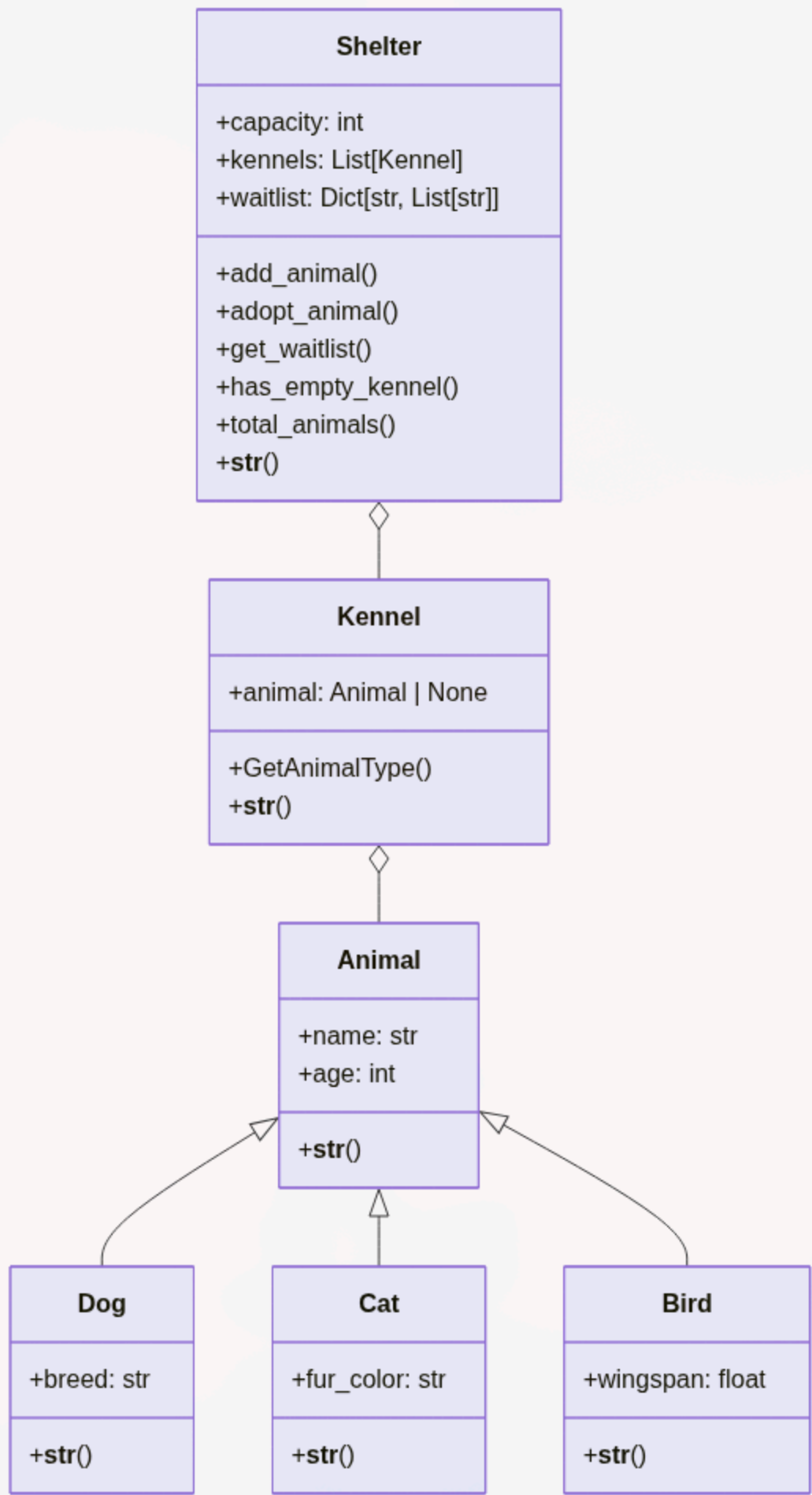
This follows the "is-a" relationship: every Dog is an Animal, every Cat is an Animal, and every Bird is an Animal. The base `Animal` class provides common attributes (name and age), while each subclass adds its own specific attributes.

Composition Relationships

The system uses composition to build complex structures from simpler components:

1. **Kennel** → **Animal**: Each kennel composes exactly one animal (or `None` if empty), following the "**has-a**" relationship
2. **Shelter** → **Kennels**: The shelter composes a list of kennels, bounded by its capacity limit
3. **Shelter** → **Waitlist**: The shelter maintains a dictionary mapping animal types to lists of adopter names

UML Object Model



Relationship Summary

From Class	To Class	Relationship Type	Cardinality	Description
Dog	Animal	Inheritance	1:1	Dog extends Animal with breed attribute
Cat	Animal	Inheritance	1:1	Cat extends Animal with fur_color attribute
Bird	Animal	Inheritance	1:1	Bird extends Animal with wingspan attribute
Kennel	Animal	Composition	1:0..1	Kennel holds zero or one animal
Shelter	Kennel	Composition	1:0..n	Shelter contains 0 to capacity kennels
Shelter	Waitlist	Composition	1:1	Shelter owns a dictionary of waitlist entries

Detailed Class Documentation

Animal (Base Class)

The `Animal` class is the base class for all animal types. It defines the common attributes that all animals share.

Attributes

Attribute	Type	Default Value	Description
name	str	""	The animal's name (can be empty for new animals)
age	int	0	The animal's age in years

Methods

- `__init__(name: str = "", age: int = 0) -> None` : Constructor with default parameters allowing flexible instantiation
- `__str__() -> str` : Returns formatted string "Name: {name}, Age: {age}"

Design Notes

The base class uses empty string and zero as defaults instead of requiring parameters. This allows:

1. Animals to be created first and populated later
2. Test cases to quickly create placeholder objects

Dog Class

Extends `Animal` to represent dogs with breed information.

Additional Attributes

Attribute	Type	Default Value	Description
breed	str	""	The dog's breed (e.g., "Labrador")

Methods

- `__init__(name: str = "", age: int = 0, breed: str = "")` -> None : Calls parent constructor via `super()` and initializes breed
- `__str__()` -> str : Returns "Dog: Name: {name}, Age: {age}, Breed: {breed}"

Usage Example

```
# Create a fully specified dog
buddy = Dog("Buddy", 3, "Labrador")

# Create with partial parameters (breed defaults to "")
rex = Dog("Rex", 5)

# Create empty dog for later initialization
placeholder = Dog()
```

Cat Class

Extends `Animal` to represent cats with fur color information.

Additional Attributes

Attribute	Type	Default Value	Description
fur_color	str	""	The cat's fur color (e.g., "Tabby")

Methods

- `__init__(name: str = "", age: int = 0, fur_color: str = "") -> None` : Initializes cat with inherited and specific attributes
- `__str__() -> str` : Returns "Cat: Name: {name}, Age: {age}, Fur Color: {fur_color}"

Bird Class

Extends `Animal` to represent birds with wingspan measurement.

Additional Attributes

Attribute	Type	Default Value	Description
wingspan	float	0.0	The bird's wingspan in meters

Methods

- `__init__(name: str = "", age: int = 0, wingspan: float = 0.0) -> None` : Initializes bird including floating-point wingspan
- `__str__() -> str` : Returns "Bird: Name: {name}, Age: {age}, Wingspan: {wingspan}"

Note: The `Bird` class is the only animal type using a `float` attribute.

Kennel Class

The `Kennel` class is a container for a single animal. It enforces the rule that each kennel can hold at most one animal at a time.

Attributes

Attribute	Type	Default Value	Description
animal	Optional[Animal]	None	The current occupant (None if empty)

Methods

Method	Return Type	Description
<code>__init__(animal=None)</code>	None	Creates a kennel, optionally with an initial animal
<code>GetAnimalType()</code>	str	Returns class name of animal ("None" if empty)

Method	Return Type	Description
<code>__str__()</code>	<code>str</code>	Returns "Kennel(empty)" or "Kennel Animal: {animal_str}"

Single Occupancy Rule

When assigning a new animal to a kennel that already contains one, the previous animal is replaced (not added alongside). This reflects the constraint that physical kennels have limited space:

```
kennel = Kennel(Dog("Buddy", 3, "Labrador"))
kennel.animal = Cat("Mittens", 2, "Tabby") # Buddy is replaced, not stored
```

Type Identification

The `GetAnimalType()` method uses Python's reflection capability to identify the animal's class:

- Returns "Dog", "Cat", or "Bird" for occupied kennels
- Returns "None" for empty kennels
- Implementation: `self.animal.__class__.__name__`

Shelter Class

The `Shelter` class manages the animal shelter system. It maintains a collection of kennels, enforces capacity limits, processes adoptions, and tracks unfulfilled requests through a waitlist.

Attributes

Attribute	Type	Description
<code>capacity</code>	<code>int</code>	Maximum number of kennels allowed (must be positive)
<code>kennels</code>	<code>List[Kennel]</code>	Dynamic list of kennels (grows from 0 to capacity)
<code>waitlist</code>	<code>Dict[str, List[str]]</code>	Maps animal types to lists of adopter names

Constructor Validation

The `__init__` method enforces that capacity must be positive:

```
def __init__(self, capacity: int) -> None:
    if capacity <= 0:
        raise ValueError("Shelter capacity must be positive.")
    # ...initialization continues...
```

This prevents nonsensical configurations like shelters with zero or negative capacity.

Methods Overview

Method	Return Type	Purpose
add_animal(animal)	bool	Intake animal; returns success status
adopt_animal(animal_type, adopter_name)	Optional[Animal]	Process adoption request
get_waitlist(animal_type)	List[str]	Retrieve waitlist for a type
has_empty_kennel()	bool	Check if any kennel is available
total_animals()	int	Count current animal population
_normalize_type(animal_type) (static)	str	Standardize type names for comparison

Algorithm Design and Workflows

Animal Intake Workflow (add_animal)

The intake process follows a two-phase strategy to maximize kennel utilization:

Phase 1: Kennel Reuse

```
FOR each existing kennel IN kennels:
    IF kennel.animal IS None:
        ASSIGN animal to kennel
        RETURN True (success)
```

This phase scans existing kennels to find empty slots before creating new kennels. This minimizes the number of physical kennels needed.

Phase 2: Kennel Expansion

```
IF length of kennels < capacity:
    CREATE new Kennel containing the animal
    APPEND new kennel to kennels list
    RETURN True (success)
ELSE:
    RETURN False (shelter at capacity)
```

If no empty kennels exist, the shelter creates a new one (but only if it hasn't reached capacity). This enforces the hard limit on shelter size.

Example Scenario

```
shelter = Shelter(capacity=3)
shelter.add_animal(Dog("Buddy", 3, "Labrador")) # Creates kennel #1
shelter.add_animal(Cat("Mittens", 2, "Tabby")) # Creates kennel #2
shelter.adopt_animal("Cat") # Frees kennel #2
shelter.add_animal(Bird("Sky", 1, 0.35)) # Reuses kennel #2 (no
new kennel created)
```

Adoption Processing Workflow (adopt_animal)

The adoption system handles both successful matches and unfulfilled requests:

Step 1: Type Normalization

```
normalized_type = _normalize_type(animal_type)
```

Convert requested type to lowercase and trim whitespace. This allows case-insensitive matching ("dog", "Dog", "DOG" all match).

Step 2: Kennel Search

```
FOR each kennel IN kennels:
    IF kennel contains an animal AND animal's type matches normalized_type:
        STORE reference to animal
        SET kennel.animal to None (free the kennel)
    RETURN the animal
```

The first matching animal is selected. Once found, its kennel is immediately freed for future intake.

Step 3: Waitlist Recording (if no match found)

```
IF adopter_name was provided:  
    ADD adopter_name to waitlist[normalized_type]  
RETURN None (no animal available)
```

When no matching animal exists and an adopter name is provided, the system records this unfulfilled request for future follow-up.

Adoption Return Values

- **Success:** Returns the `Animal` object that was adopted
- **Failure (no animal):** Returns `None`
- **Failure (at capacity):** Returns `None`

Waitlist Management (`get_waitlist`)

The waitlist system tracks people who requested animals that weren't available:

```
FUNCTION get_waitlist(animal_type):  
    normalized = _normalize_type(animal_type)  
    RETURN copy of waitlist[normalized] (or empty list if key doesn't exist)
```

Important Note: The method returns a *copy* of the waitlist, not the internal list. This prevents external code from modifying the shelter's internal state:

```
waitlist = shelter.get_waitlist("Rabbit")  
waitlist.append("Eve") # This ONLY modifies the copy, not shelter's  
internal list
```

Type Normalization (`_normalize_type`)

This static utility method ensures consistent type comparisons throughout the system:

```
@staticmethod  
def _normalize_type(animal_type: str) -> str:  
    return animal_type.strip().lower()
```

Why Static? This method doesn't need access to instance data, so it's marked `@staticmethod`. This means it's a utility function that belongs to the class but doesn't depend on instance state.

Normalization Steps:

1. `.strip()` : Removes leading/trailing whitespace
2. `.lower()` : Converts to lowercase

Examples:

- `" Dog "` → `"dog"`
- `"CAT"` → `"cat"`
- `"bird"` → `"bird"`

Type System and Annotations

The codebase uses Python's `typing` module to provide clear contracts and improve code maintainability. These annotations serve as documentation and enable static type checking tools.

Type Imports Used

```
from typing import Dict, List, Optional, Union
```

Type Annotation Reference

Annotation	Meaning	Used In
<code>str</code>	String value	All name/type parameters
<code>int</code>	Integer value	age, capacity
<code>float</code>	Floating-point number	Bird wingspan
<code>bool</code>	Boolean (True/False)	Return value of <code>add_animal</code>
<code>None</code>	No value	Constructor return types
<code>Optional[X]</code>	Either type X or None	<code>Kennel.animal</code> , <code>adopt_animal</code> return
<code>List[X]</code>	List containing elements of type X	<code>kennels</code> , <code>waitlist</code> values
<code>Dict[K, V]</code>	Dictionary with keys of type K, values of type V	<code>waitlist</code>
<code>Union[A, B, C]</code>	One of several types	Animal type alias

Type Alias Pattern

The code defines a type alias to simplify references to animal types:

```
Animal = Union[Dog, Cat, Bird]
```

This allows code to declare "Animal" as a type, which the type checker understands as "Dog or Cat or Bird". This appears in:

- Kennel.__init__(animal: Optional[Animal])
- Shelter.add_animal(animal: Animal)
- Shelter.adopt_animal() -> Optional[Animal]

Sample Data and Test Scenarios

Test Animals Used in Demonstration

Variable	Type	Name	Age	Type-Specific Attribute	Value
dog1	Dog	Buddy	3	breed	Labrador
cat1	Cat	Mittens	2	fur_color	Tabby
bird1	Bird	Sky	1	wingspan	0.35 (meters)

Adoption Scenario: Request for "Rabbit" with adopter name "Alice" (animal type not available in shelter)

Shelter Configuration: capacity=3 (can hold up to 3 kennels)

Complete Execution Walkthrough

The `__main__` block in `animal_shelter.py` demonstrates the full system lifecycle:

Step 1: Shelter Initialization and Animal Intake

```
shelter = Shelter(capacity=3)
shelter.add_animal(dog1)      # Kennel #1 created
shelter.add_animal(cat1)     # Kennel #2 created
print(shelter)
```

Output: Shelter(capacity=3, kennels=2, animals=2, Kennel Animal: Dog: Name: Buddy, Age: 3, Breed: Labrador, Kennel Animal: Cat: Name: Mittens, Age: 2, Fur Color: Tabby)

Step 2: Adoption Processing

```
adopted_cat = shelter.adopt_animal("Cat")
print(f"Adopted: {adopted_cat}")
print(f"Empty kennel available? {shelter.has_empty_kennel()}")
```

Output:

```
Adopted: Cat: Name: Mittens, Age: 2, Fur Color: Tabby
Empty kennel available? True
```

The adoption frees kennel #2, making it available for reuse.

Step 3: Kennel Reuse Demonstration

```
shelter.add_animal(bird1)    # Reuses kennel #2 (no new kennel created)
print(shelter)
```

Output: Shelter(capacity=3, kennels=2, animals=2, Kennel Animal: Dog: Name: Buddy, Age: 3, Breed: Labrador, Kennel Animal: Bird: Name: Sky, Age: 1, Wingspan: 0.35)

Notice the kennel count remains at 2 because the bird occupies the previously freed kennel.

Step 4: Waitlist Tracking

```
rabbit_request = shelter.adopt_animal("Rabbit", adopter_name="Alice")
print(f"Rabbit adoption result: {rabbit_request}")
print(f"Rabbit waitlist: {shelter.get_waitlist('Rabbit')}")
```

Output:

```
Rabbit adoption result: None
Rabbit waitlist: ['Alice']
```

Since no rabbit exists in the shelter, Alice is added to the waitlist for future notification.

Test Strategy

The test suite in `test_animal_shelter.py` contains **33 unit tests** organized into five test classes, each targeting a specific component of the system.

Test Suite Structure

```
test_animal_shelter.py (33 tests)
├─ TestDog (4 tests)
├─ TestCat (4 tests)
├─ TestBird (4 tests)
├─ TestKennel (16 tests)
└─ TestShelter (5 tests)
```

Test Class: TestDog (4 tests)

Validates the Dog class constructor overloading and string representation.

Test Name	Purpose
<code>test_dog_constructor_with_parameters</code>	Verifies Dog stores name, age, and breed correctly
<code>test_dog_constructor_default</code>	Tests parameterless constructor (all defaults)
<code>test_dog_str</code>	Validates <code>__str__</code> format: "Dog: Name: X, Age: Y, Breed: Z"
<code>test_dog_partial_parameters</code>	Confirms partial initialization (e.g., name and age only)

Key Testing Technique: Each test follows the **Arrange-Act-Assert** pattern:

1. **Arrange:** Create the object with specific parameters
2. **Act:** (Usually implicit—constructor runs)
3. **Assert:** Verify attributes match expected values

Test Class: TestCat (4 tests)

Mirrors the Dog tests but validates Cat-specific attributes (`fur_color` instead of `breed`).

- Tests same scenarios as Dog class
- Validates format: "Cat: Name: X, Age: Y, Fur Color: Z"
- Ensures inheritance from Animal works correctly

Test Class: TestBird (4 tests)

Validates the Bird class with its unique float attribute (wingspan).

- Tests floating-point attribute handling
- Validates format: "Bird: Name: X, Age: Y, Wingspan: Z"
- Confirms default of 0.0 for wingspan

Special Consideration: Bird tests verify that Python correctly handles float default values.

Test Class: TestKennel (16 tests)

The most comprehensive test class, divided into five logical sections:

Section 1: Constructor Tests (4 tests)

- `test_kennel_constructor_empty` : Verify empty kennel creation
- `test_kennel_constructor_with_dog` : Initialize with Dog
- `test_kennel_constructor_with_cat` : Initialize with Cat
- `test_kennel_constructor_with_bird` : Initialize with Bird

These tests ensure all three animal types can be stored in kennels.

Section 2: Adding Animals (3 tests)

- `test_kennel_add_dog` : Add dog to empty kennel
- `test_kennel_add_cat` : Add cat to empty kennel
- `test_kennel_add_bird` : Add bird to empty kennel

Section 3: Critical Business Rule Test (1 test)

- `test_kennel_replace_animal` : **Most important kennel test**

This test verifies that kennels hold only ONE animal at a time. When a new animal is assigned, it replaces the previous one:

```
kennel.animal = dog
self.assertEqual(kennel.animal, dog)
kennel.animal = cat # Replacement occurs here
self.assertEqual(kennel.animal, cat)
self.assertNotEqual(kennel.animal, dog) # Dog no longer in kennel
```

Section 4: GetAnimalType() Method Tests (4 tests)

- `test_kennel_get_animal_type_empty` : Returns "None" for empty kennel
- `test_kennel_get_animal_type_dog` : Returns "Dog" for dog

- `test_kennel_get_animal_type_cat` : Returns "Cat" for cat
- `test_kennel_get_animal_type_bird` : Returns "Bird" for bird

Testing Technique: These tests verify Python reflection (`__class__.__name__`) works correctly.

Section 5: String Representation Tests (4 tests)

- `test_kennel_str_empty` : Empty kennel displays as "Kennel(empty)"
- `test_kennel_str_with_dog` : Kennel with dog shows full animal details
- `test_kennel_str_with_cat` : Kennel with cat shows full animal details
- `test_kennel_str_with_bird` : Kennel with bird shows full animal details

Test Class: TestShelter (5 tests)

Validates the complete shelter management system introduced in Project 3.

Test Name	Validates
<code>test_shelter_capacity_must_be_positive</code>	Constructor rejects capacity ≤ 0 (raises <code>ValueError</code>)
<code>test_shelter_reuses_empty_kennel_before_creating_new</code>	Kennel reuse algorithm works correctly
<code>test_shelter_enforces_capacity_limit</code>	Cannot exceed capacity (<code>add_animal</code> returns <code>False</code>)
<code>test_shelter_adopt_removes_animal</code>	Adoption frees kennels and returns correct animal
<code>test_waitlist_records_missing_animal_requests</code>	Waitlist stores adopter names; returns copy (not reference)

Detailed Test Analysis: Kennel Reuse

```
def test_shelter_reuses_empty_kennel_before_creating_new(self):
    shelter = Shelter(2)  # Capacity: 2 kennels max

    shelter.add_animal(dog)    # Creates kennel #1
    shelter.add_animal(cat)    # Creates kennel #2 (at capacity)

    adopted_cat = shelter.adopt_animal("Cat")  # Frees kennel #2
    self.assertTrue(shelter.has_empty_kennel())  # Verify empty kennel
```

```
exists
```

```
shelter.add_animal(bird) # Should reuse kennel #2, NOT create kennel #3
self.assertEqual(shelter.total_animals(), 2) # 2 animals in 2 kennels
```

This test proves the shelter reuses empty kennels rather than wastefully creating new ones.

Detailed Test Analysis: Waitlist Immutability

```
def test_waitlist_records_missing_animal_requests(self):
    shelter = Shelter(1)
    shelter.adopt_animal("Rabbit", adopter_name="Alice")

    # Get waitlist and try to mutate it
    returned_waitlist = shelter.get_waitlist("Rabbit")
    returned_waitlist.append("Bob") # Modify the returned list

    # Verify internal list wasn't affected
    self.assertEqual(shelter.get_waitlist("Rabbit"), ["Alice"])
```

This test ensures `get_waitlist()` returns a copy of the list, not the original. This prevents external code from modifying the shelter's internal data.

Running the Test Suite

Command: `python -m unittest test_animal_shelter.py -v`

The `-v` (verbose) flag provides detailed output showing each test as it executes:

```
test_dog_constructor_default (test_animal_shelter.TestDog) ... ok
test_dog_constructor_with_parameters (test_animal_shelter.TestDog) ... ok
test_dog_partial_parameters (test_animal_shelter.TestDog) ... ok
test_dog_str (test_animal_shelter.TestDog) ... ok
...
-----
Ran 33 tests in 0.002s

OK
```

Test Coverage Summary

Component	Tests	Coverage Areas
Animal Base	12	Constructor defaults, inheritance, string formatting

Component	Tests	Coverage Areas
Kennel	16	Single occupancy, type identification, all animal types
Shelter	5	Capacity, kennel reuse, adoption, waitlist
Total	33	Complete system coverage

Implementation Quality Features

Defensive Programming Patterns

- 1. **Input Validation:** Shelter constructor rejects invalid capacity
- 2. **Immutable Returns:** `get_waitlist()` returns copies, not references
- 3. **Type Safety:** Type annotations help prevent errors
- 4. **Single Responsibility:** Each class has one clear purpose

Data Structure Choices

Structure	Purpose	Rationale
<code>List[Kennel]</code>	Ordered kennel collection	Supports sequential search, dynamic growth
<code>Dict[str, List[str]]</code>	Waitlist by animal type	O(1) lookup by type, lists preserve order
<code>Optional[Animal]</code>	Nullable animal references	Explicit modeling of "may not exist" cases

Conclusion

This animal shelter simulator implements object-oriented principles in Python. The system handles real-world constraints like capacity limits and resource reuse, maintains data integrity through defensive copying and validation, and includes test coverage for reliability. Adding new animal types only requires creating a new subclass of `Animal` with its specific attributes.